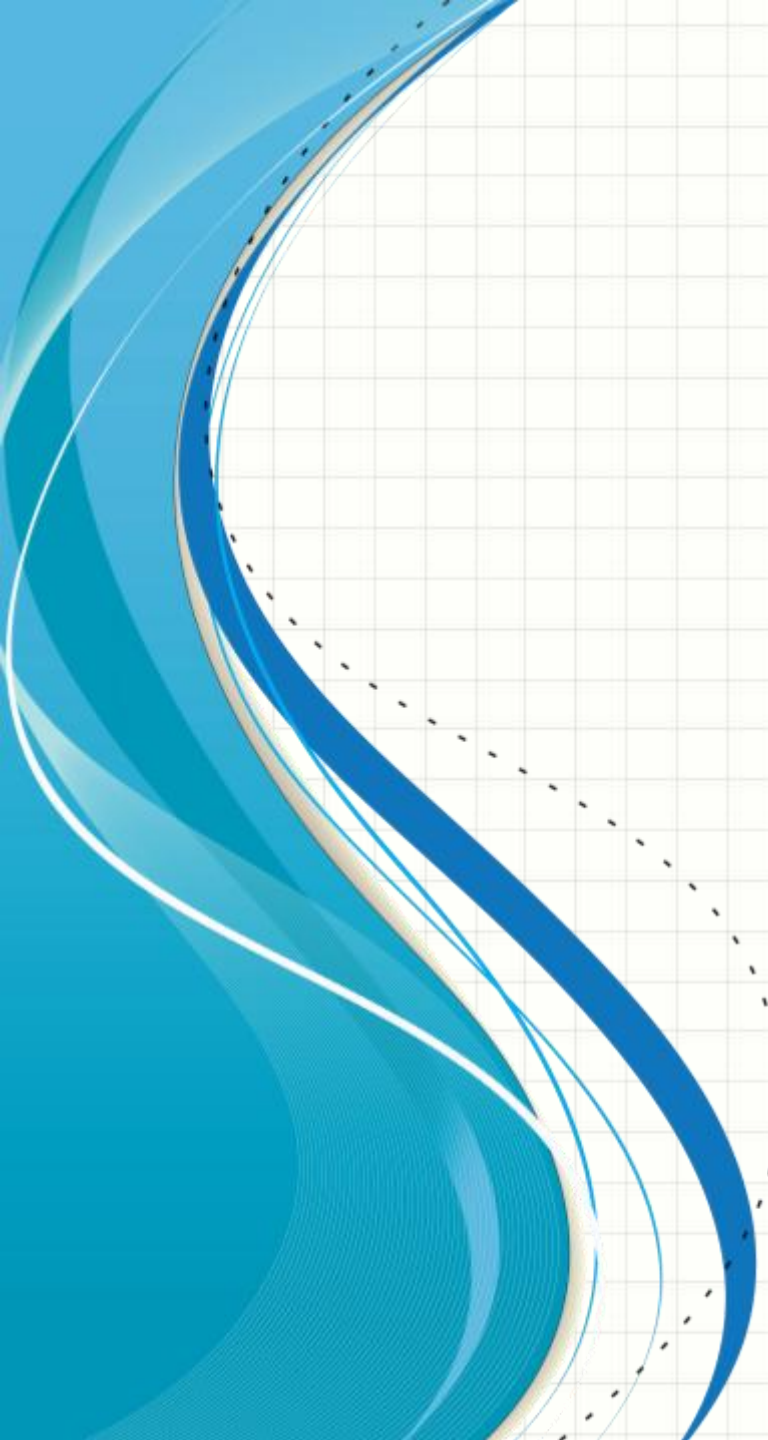




ILLINOIS INSTITUTE
OF TECHNOLOGY

Transforming Lives. Inventing the Future.

www.iit.edu

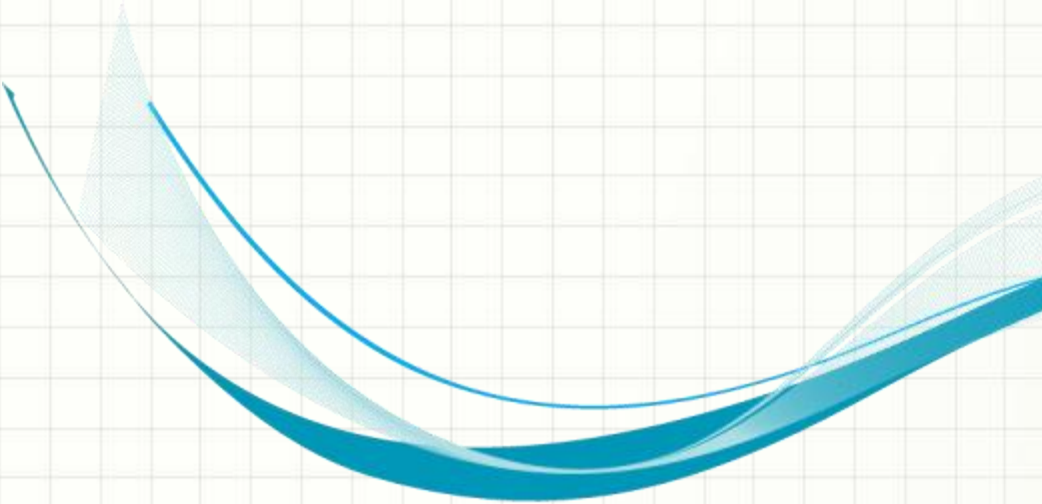
SOFTWARE ENGINEERING

CS 487

Prof. Dennis Hood
Computer Science

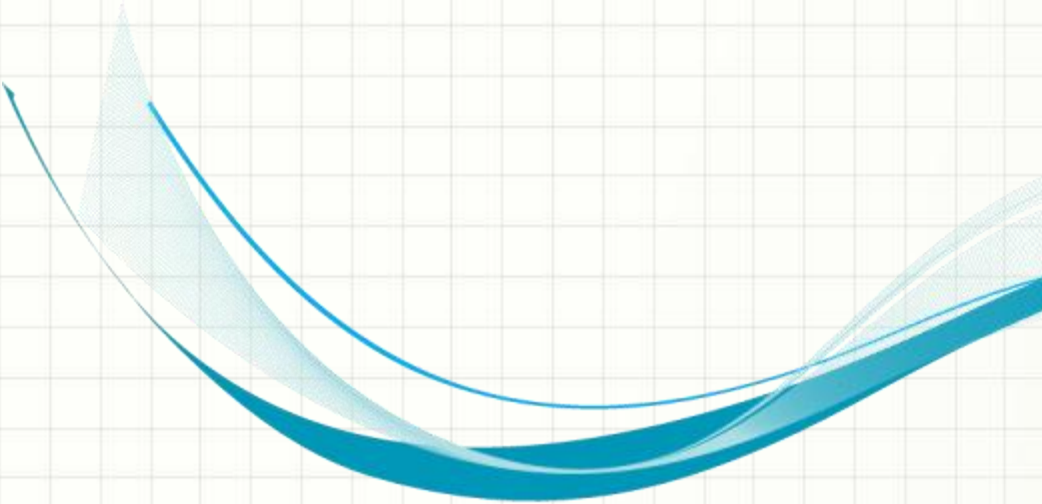
Lesson Overview

- Human Awareness Design
- Reading
 - Ch. 22 – Project Management
- Objectives
 - Discuss the discipline of project management with respect to the software engineering life cycle
 - Explore the awareness with respect to human partners, both direct and indirect
 - Examine design approaches for supporting minimal protocols
 - Define situational awareness (SA) as a measure of the effectiveness of HCI design
 - Analyze design requirements which pose significant challenges to SA



Week 13

Human Awareness Design



Project Management

What is a Project?

- A project is the application of resources to produce a product / deliver a service
 - Oversees a software engineering macro-iteration
- Projects are temporary and unique
 - Temporary implies a definite beginning and ending
 - Unique implies that it has never been done before
 - Projects have objectives / requirements
- A project begins when the involved parties agree to the objectives
 - Contractual obligation; the start of a macro-iteration
- A project ends when the objectives (or an agreed upon revised set) are met or are determined unattainable
 - Successful user-acceptance test (UAT)

Objectives and Constraints

- Objectives are desired results and form the basis of project planning
 - The project plan is designed to achieve the project's objectives
 - The project is done when the objectives have been met (generally speaking)
- Constraints
 - Restrictions on the structure of the project primarily associated with the parameters of the scope triangle

Stakeholders

- A stakeholder is anyone impacted by the project and/or its outcome
 - Customer
 - Sponsor
 - Management
 - Project team
 - Partners, etc.
- Proactive communication is critical to the stakeholders' satisfaction

The Need for Project Definition

- By “definition” we mean establishing the purpose and parameters of the project
 - What will this project achieve?
 - When will it complete?
 - How many people?, having which skills?
 - How much will it cost?
- Consensus among stakeholders is critical to success
- Definition must precede planning

Scope of Work to be Performed

- Scope defines the objectives and the work to be produced
 - Begins as a client request and is negotiated into a contract of deliverables
 - Must also define what won't be done
 - Establishes key milestones and environmental requirements
 - Also agree to conditions of satisfaction
 - Project scope vs. product scope
 - Largely overlaps with software engineering's requirements

Inter-related Project Parameters

- What we can accomplish (*scope*) and how well we do it (*quality*) are factors of:
 - How much we invest (*cost*)
 - How long we take (calendar *time*)
 - And how hard we work (resource *effort*)
- A change to any of these 5 factors forces a change to at least 1 of the other 4

Managing Project Priorities

- Priorities determine which factors are:
 - Constrained / Fixed – cannot be altered
 - Enhance – should be optimized
 - Accept / Tolerate – can be slackened
- Factors must be negotiated to meet the objectives of the project while still maintaining the equilibrium of the scope triangle

Work Decomposition

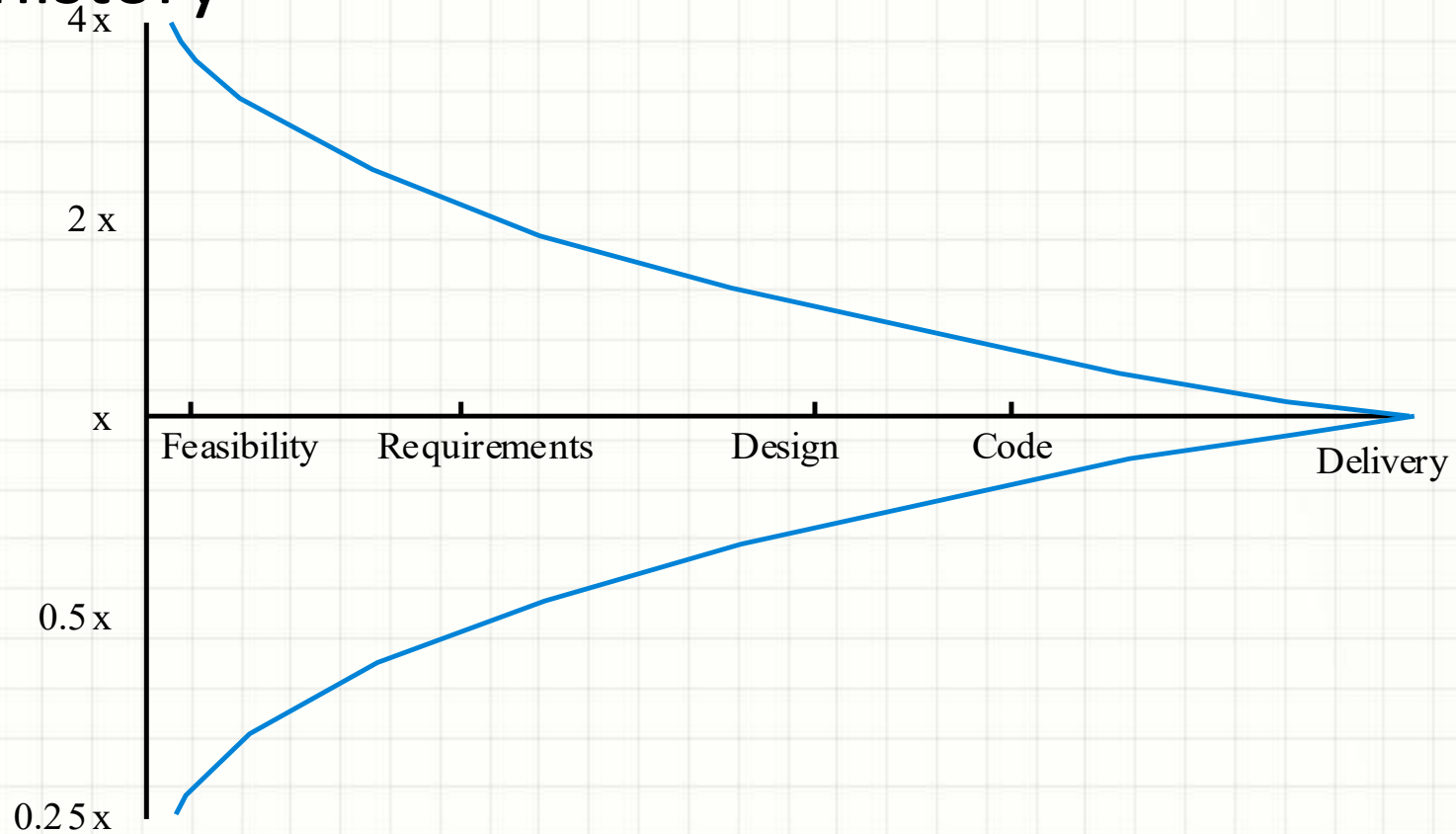
- Breaking the goal into manageable pieces
- Decomposition techniques
 - Top-down vs. bottom-up
 - Activities vs. deliverables
- Stopping Rules
 - Progress and completion can be measured
 - Start and end events are clearly defined
 - Effort can be estimated with confidence
 - Required skills are well understood

Estimating Software Development

- Things to estimate/predict
 - Time, effort, cost, etc.
 - Defects, user satisfaction, etc.
- What drives these?
 - Requirements, objects, components, subsystems, interfaces, methods, data tables, complexity, others?
- Cost elements
 - Labor
 - Infrastructure
 - Tools
 - Travel, training, etc.

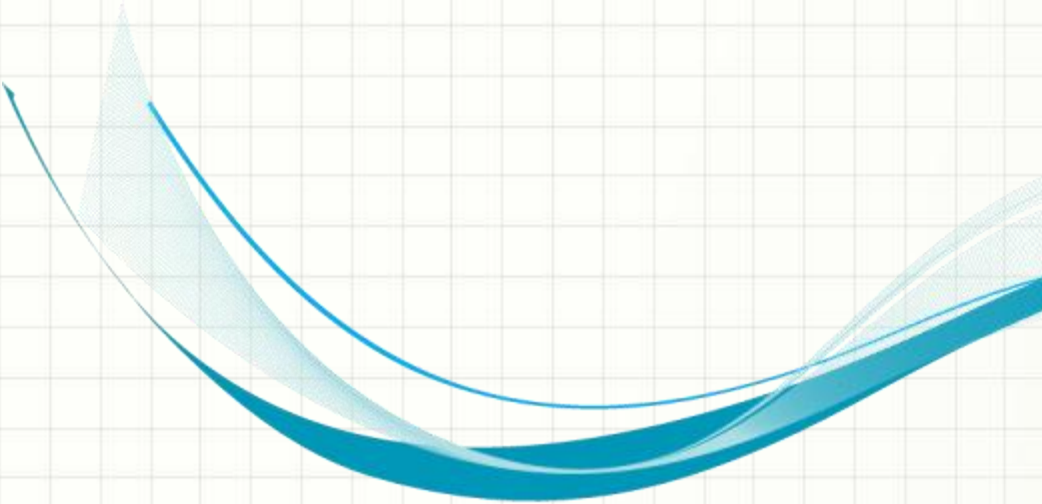
Estimation

- We predict the future by utilizing relevant past history



Productivity

- N units produced / M person-hours
- What should the numerator refer to in software engineering efforts?
 - Many possible adequate solutions, each potentially with many ways to achieve it
 - Size-related such as lines of code
 - Function-related such as function points
- Factors affecting productivity
 - Experience
 - Process maturity
 - Project size
 - Tool support
 - Working environment



Human Awareness Engineering

Human Awareness Defined

- Being aware of your environment
 - Current state
 - What's coming next
- Able to extract relevant information
 - The flow of information
 - Prioritization
- Awareness is assessed with respect to the specific goals of a particular job
- Human partners
 - Directly interacting with our system
 - Indirectly effected by our system

Mission Critical Awareness Examples

- A driver operating a vehicle
 - Presence of other vehicles and obstacles
 - Speed and direction
 - Rules and condition of the road
- A doctor treating a patient
 - Vital signs and symptoms
 - Relevant patient history
 - “Best practices”
- An air traffic controller managing flights
 - Speed, direction and altitude of all planes
 - Weather conditions
- Note that each of these could be automated
 - Is it worth it?

The Importance of Awareness

- Awareness improves decision making
 - Faster
 - Better
- Awareness prepares the user for “exceptional”
 - The universe of exceptions is smaller and narrower and is therefore more manageable
- Awareness facilitates appropriate automation
 - “normal” is predictable and can therefore be automated safely

Situation Awareness (SA) Levels

- Level 1 – Perception
 - Status, attributes, and dynamics of relevant elements
- Level 2 – Comprehension
 - What does the data mean in the context of relevant goals and objectives
- Level 3 – Projection
 - Predicting future state
 - Selection of next action(s)

Time Lags

- From unaware to taking action
 - Level 1 – receive an indication
 - Level 2 – process to recognize its meaning
 - Level 3 – assess the proper response based on previously obtained knowledge and/or analysis
- Communication, processing, retrieval, assessment, and execution all take time
- The duration between an event happening and a return to normal (or at least safe?) is the total time lag
- Minimizing time lag is highly desirable; beating the “point of no return” time is required
- Making time lag consistent and therefore predictable allows for a complete engineered solution (automation?)

The Awareness Loop

- An entity (H or C) gains awareness through sensing followed by analyzing / processing
- One entity can “inspire” the awareness of a partner by sending a signal
- But, importantly, the sender cannot guarantee the receiver actually becomes (sufficiently) aware
- The loop can be closed by the receiver replying with an acknowledgement of its awareness
- The time to close the loop should be included in the total time lag

Assessment Elements

- Time
 - How long does it take to become situational-ly aware
- A product of the process
 - Users become situational-ly aware through a process of investigation and analysis
- Perception and attention
 - Humans have finite capacities for both conscious and sub-conscious processing
 - Context, previous experiences, time, etc.
- Working memory
 - Have I seen / experienced this before?
 - 7 +/- 2 chunks of information in “working” memory
- Mental models
 - Users organize experiences and analyze situations methodically using models proven to be effective
- User goals
 - “Did I get my job done?”

Dashboards

- Information of all kinds can be consolidated into dashboard imagery
- Dashboards have become quite popular
 - Does that mean they are effective?, or just familiar?
 - Will popularity help make them effective?
- Historical precedence
 - Car dashboards
 - Industrial control panels
- Business intelligence – so simple a CEO can use it
- Enron fallout – liability of not conveying knowledge to shareholders

Sources of Uncertainty

- SA uncertainty
 - Perception of elements (level 1)
 - Missing (e.g., hidden) information
 - Reliability/credibility of data
 - Conflicting data
 - Timeliness
 - Noisy data
 - Comprehension confidence (level 2)
 - Challenging abstraction
 - Will decisions lead to desired outcomes? (level 3)
- Decision uncertainty is a factor of the uncertainty the user has across the 3 levels of SA

Managing Uncertainty

- Users naturally work to remove uncertainty
 - Rely on defaults
 - Perfection may not be necessary
 - Process of elimination
- Good design helps by
 - Providing supplemental information
 - Representing critical information in multiple ways
 - Provide multiple sources to help resolve conflicts
 - Supporting “bet-hedging”

Support Uncertainty Management

- Make it easy for users to assess reliability
 - Deciding among several close alternatives takes time
 - Fewer, more distinct choices means more rapid decision making
- Cues to assist should be prominently presented proximally to the display information
 - Confirming or negating information
 - Contextual information

Complexity and SA

- Complexity inherently impedes SA
- However complexity is dynamic
 - The more we learn about a complex environment, the simpler it becomes
 - Of course, the demand for more features increases the complexity again
- Designers' goals
 - Balance features and complexity
 - Help users learn and get comfortable
 - Expand the feature set judiciously

System Complexity

- System complexity is relatively objective
 - System complexity can be measured
 - And therefore reduced in a measurable fashion
- Metrics
 - Number of items
 - Objects, functions, etc.
 - Degree of interaction among items
 - System dynamics
 - Frequency of status change
 - Predictability of changes

Operational Complexity

- How difficult is the system to use?
- System complexity does not imply operational complexity (e.g., cars)
 - Reducing operational complexity can offset and even override system complexity
- Each user category will have its own rating of operation complexity
- Automation can reduce operational complexity, but not necessarily

Apparent Complexity

- Apparent complexity is relatively subjective
 - Does the system “look” complex?
 - Basically an assessment of the UI
 - A bad UI can make a simple system complex
- A function of
 - Cognitive complexity
 - Ease of accurately mapping interface to operation
 - Transparency or surfacing system status
 - Display complexity (density, grouping, etc.)
 - Task or response complexity
 - Challenge in accomplishing goals

Complexity Design Principles

- Resist feature creep
- Manage feature group (prioritize)
- Be consistent
- Simplify flow (minimize branching)
- Analogies and metaphors
- Transparency and observability
- Group information logically
- Balance display density with coherence
- Establish and respect standards
- Minimize task complexity

The Role of Alarms

- Sometimes it's really, really important to get the user's attention
 - Something has gone wrong
 - An automated system is no longer “comfortable” being in charge
 - It's time to change course, check the latest readings, etc.
- When it's one of those times, the alarm needs to effectively grab the user's attention
 - Flashing lights
 - Annoying sounds
 - Changing colors, shapes, etc.
 - Multiple approaches may be required

Failure to Alarm

- Too many alarms
 - Information overload / confusion
 - Hard to process => easy to ignore
- False alarms
 - Users react with complacency or even ignore the alarms (crying wolf)
- Intentional disarming
 - Users sometimes disable alarms
 - Often due to #1 and #2 above
 - Responding to “unnecessary” alarms can cost valuable time
- We need better designed alarm systems!
 - Thresholds, triggers, sensitivity, etc.

Alarm Realities

- Studies show that in practice user's often fail to respond immediately
- Reliability
 - User's perception of past correlation between the alarm and its value
 - Designers must avoid creating “wolf-crying” systems
 - Be sensitive to environmental specifics
- Users look for confirmation
 - Supporting data, a second alarm, visual cues, etc.
- Expectations and perceptions
 - User's can become trained to expect alarms under certain circumstances
 - However, the “expected” alarm may have a different cause (than what the user perceives)

Alarm Realities (cont.)

- Disruptions and diversions
 - Clearing an alarm may take the user away from “more important” tasks
- Workload
 - Underload vs. overload, real vs. false alarms
- Alarm characteristics
 - Volume, brightness, frequency, color, melodic structure, etc.
- Diagnosis of alarms
 - Causal analysis vs. preconceived notions
- Alarm reduction schemes
 - E.g., don't alarm during start-up

Building Better Automation

- Adaptive automation
 - Periodically pulls the user in
 - When to engage the user
 - When the user seems dis-engaged
 - During critical events
- Levels of automation
 - A more appropriate mix of system and user control

Design Principles for Automation

- Only if necessary
- Routine actions, not high-level tasks
- SA support, not decisions
- Keep the user in-the-loop
- Minimize modes of automation
- Make state obvious
- Enforce consistency
- Avoid advanced queuing of tasks
- Avoid information cueing
- Human/system symbiosis
- Provide transparency

Benefits of Uninhabited Vehicles

- Boldly go where no one can go
 - Distance
 - Time
 - Oxygen
 - Pressure
- Challenging, even dangerous environments
 - Unstable terrain/buildings
 - Combat areas
- Maintaining secrecy and safety
 - Distance from the enemy
- Automation

Challenges

- Somebody still has to drive the thing
 - Start/stop, left/right, up/down, etc.
 - Commands transmitted via specified medium
- The operator is acting without true sensory feedback and simulated feedback has many shortcomings
 - Sluggish response to commands
 - Delays in receiving feedback
 - Missing or misinformation
 - Lack of “big picture”
 - Lack of subtle cues (or at least delay)
- Relatively immature technology
- Absence of risk can lead to carelessness